

BaseKit Interview Prep Guide

PHP Backend Developer — Round 2

PREPARED FOR
Interview Preparation

BY
Chris Garlick

DATE
2026-06-11

About This Guide

A focused refresher for the BaseKit PHP Backend Developer — Round 2 interview. Each section opens with why it matters, then gives the concept in plain English and a PHP example you can speak to. Read it once end-to-end, then drill the topics that came up in round one.

CORE LANGUAGE

1. PHP 8 vs PHP 7
2. Dependency Injection
3. SOLID Principles
4. OOP Concepts

DATA & APIs

4. MySQL Performance
5. REST API Design
6. Security Essentials

TOOLING & CRAFT

6. Docker & Kubernetes
7. Linux / Mac CLI
8. Git Workflow
9. Best Practices

How to use it: skim the headings, read every Why it matters box, and rehearse saying each idea out loud. The phrases on the final page are the ones to drop into answers.

1 · PHP 8 Features vs PHP 7

WHY IT MATTERS

PHP 8 (Nov 2020) was a major release — big performance gains and real language features, built on by 8.1 and 8.2. Coming from WordPress / PHP 7, these are the things you need to know cold.

Named Arguments · 8.0

Pass arguments by name, in any order, skipping optional ones.

```
// PHP 7 – positional only
array_slice($array, 0, 5, true);

// PHP 8 – named, readable, order doesn't matter
array_slice(array: $array, offset: 0, length: 5, preserve_keys: true);
```

Union Types · 8.0

Declare that a value can be one of several types.

```
function processInput(int|string $input): int|string {
    return $input;
}
```

Nullsafe Operator · 8.0

Collapses nested null checks — if any link is null, the chain returns null.

PHP 7

```
$country = null;
if ($user !== null) {
    if ($user->getAddress() !== null) {
        $country = $user->getAddress()->getCountry();
    }
}
```

PHP 8

```
$country = $user
    ?->getAddress()
    ?->getCountry();
```

Match Expressions · 8.0

Like `switch`, but it returns a value, uses strict `===` comparison, and has no fall-through.

```
$label = match($status) {  
    1 => 'Active',  
    2 => 'Inactive',  
    default => 'Unknown',  
};
```

A `match` with no arm and no `default` throws `UnhandledMatchError` — safer than `switch` silently doing nothing.

Constructor Property Promotion · 8.0

Declare and assign properties in one line — great for DTOs and value objects.

```
class User {  
    public function __construct(  
        private string $name,  
        private string $email,  
    ) {}  
}
```

Readonly Properties · 8.1

Set once at initialisation, immutable thereafter.

```
class Order {  
    public function __construct(  
        public readonly int $id,  
        public readonly string $status,  
    ) {}  
}  
  
$order = new Order(1, 'pending');  
$order->id = 2; // Error: Cannot modify readonly property
```

Enums · 8.1

Native enumerations — no more class constants or magic strings.

```
enum Status: string {
    case Active = 'active';
    case Inactive = 'inactive';
}

function setStatus(Status $status): void { /* ... */ }
setStatus(Status::Active);

$status = Status::from('active'); // Status::Active
```

FIBERS · 8.1

Lightweight concurrency — pause and resume execution without a full event loop. Know what they are and why they exist; you won't be asked to write one.

FIRST-CLASS CALLABLES · 8.1

Cleaner way to pass a function as a callable:

```
$fn = strlen(...);
array_map(strlen(...),
    $strings);
```

Intersection Types · 8.1 & readonly Classes · 8.2

Union = this OR that; intersection = this AND that (must satisfy all). A `readonly` class makes every property readonly by default.

```
function process(Iterator&Countable $collection): void { /* ... */ }

readonly class Point {
    public function __construct(public float $x, public float $y) {}
}
```

2 · Dependency Injection

WHY IT MATTERS

BaseKit's job ads call out DI as a core practice, and they use it heavily on the React frontend too. Being able to articulate why DI is good — testability, loose coupling, single responsibility — matters as much as the how.

Plain English: instead of a class creating its own dependencies, you pass them in from outside. The class declares what it needs; something else provides it.

Like a restaurant kitchen — the chef doesn't go to the farm, ingredients are delivered. The chef just cooks.

The problem it solves

✗ WITHOUT DI

Tightly coupled, hard to test.

```
class OrderService {
    public function
    __construct() {
        $this->db = new
        MySQLDatabase();
        $this->mailer =
        new SmtplibMailer();
    }
    // Can't test without a
    // real DB + SMTP
```

✓ WITH DI

Loosely coupled, testable.

```
class OrderService {
    public function
    __construct(
        private
        DatabaseInterface $db,
        private
        MailerInterface $mailer,
    ) {}
    // Tests pass in fakes/
    // mocks
```

Three types of injection

CONSTRUCTOR

Most common — required upfront.

```
public
function
__construct(
    private
    LoggerInterface
    $logger,
) {}
```

SETTER

Optional — set after construction.

```
public
function
setLogger(

    LoggerInterface
    $logger,
): void {
    $this->logger =
    $logger;
}
```

INTERFACE

The dependency supplies an injector method. Rare in practice.

DI Containers

Real apps wire everything together with a container — Laravel’s service container, Symfony’s DI component, or PHP-DI.

```
// Bind an interface to a concrete implementation
$container->bind(DatabaseInterface::class, MySQLDatabase::class);

// Resolving OrderService now injects a MySQLDatabase automatically
$service = $container->make(OrderService::class);
```

Say this: “DI gives me loose coupling and testability — I program to interfaces, not implementations.”

3 · SOLID Principles

WHY IT MATTERS

BaseKit mention SOLID explicitly. Know each letter and a one-line PHP example for it — the goal of all five is small, focused classes that are easy to change.

S — Single Responsibility

A class should have one, and only one, reason to change.

```
// GOOD – each concern is separate
class UserRepository { public function save(User $u): void {} }
class UserMailer    { public function sendWelcome(User $u): void {} }
class UserReporter  { public function generate(User $u): void {} }
```

O — Open/Closed

Open for extension, closed for modification — add behaviour without editing existing code.

```
interface PaymentGateway {
    public function charge(float $amount): void;
}
class StripeGateway implements PaymentGateway { /* ... */ }
class PaypalGateway implements PaymentGateway { /* ... */ }

class PaymentProcessor {
    public function process(PaymentGateway $gateway, float $amount):
void {
        $gateway->charge($amount);    // new types need no changes here
    }
}
```


L — Liskov Substitution

Subtypes must be usable anywhere their base type is, without breaking things. The classic trap: `Square extends Rectangle` forces both dimensions equal and breaks the rectangle's contract. Prefer a shared interface when behaviour differs.

```
// Square::setWidth() also sets height – a surprise side effect that
// breaks LSP.
// Instead, share an interface:
interface Shape {
    public function area(): float;
}
```

I — Interface Segregation

Don't force classes to implement methods they don't use. Many small interfaces beat one fat one.

```
interface Workable { public function work(): void; }
interface Breakable { public function takeBreak(): void; }

class HumanWorker implements Workable, Breakable { /* ... */ }
class RobotWorker implements Workable { /* robots don't need breaks
*/ }
```

D — Dependency Inversion

High-level modules shouldn't depend on low-level ones — both depend on abstractions.

```
interface DatabaseInterface { public function query(string $sql):
array; }

class OrderService {
    public function __construct(private DatabaseInterface $db) {}
}

class MySQLDatabase implements DatabaseInterface { /* ... */ }
class SQLiteDatabase implements DatabaseInterface { /* ... */ }
```

4 · MySQL Query Performance

WHY IT MATTERS

This came up in round one — expect more in round two. Lead with EXPLAIN, then talk indexes, then N+1.

EXPLAIN — the first move

Run it before optimising any query.

```
EXPLAIN SELECT * FROM orders WHERE user_id = 42;

-- MySQL 8+: actual timings, not estimates
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 42;
```

READING IT

- type — `const` → `ref` → `range` → `index` → `ALL` (worst). `ALL` = full table scan.
- key — index used (NULL = none).
- rows — estimated rows examined.
- Extra — beware `Using filesort` / `Using temporary`.

SAY THIS

First thing I'd do is run EXPLAIN and check whether we're doing a full table scan.

Indexes

A separate structure that finds rows without scanning the whole table — like a book's index. Leftmost-prefix rule governs composite indexes.

```
CREATE INDEX idx_user_status ON orders(user_id, status);
-- helps: WHERE user_id = 1 AND status = 'active'
-- helps: WHERE user_id = 1
-- NO help: WHERE status = 'active' (no leftmost prefix)
```

When indexes hurt: every index slows writes (INSERT/UPDATE/DELETE). Don't index blindly.

The N+1 Problem

One of the most common performance killers, especially in ORMs.

✗ N+1 — 501 QUERIES

```
$orders =  
Order::all(); // 1 query  
foreach ($orders as  
$order) {  
    echo $order->user-  
>name; // +1 each  
}
```

✓ EAGER LOAD — 2 QUERIES

```
$orders =  
Order::with('user')-  
>get();  
foreach ($orders as  
$order) {  
    echo $order->user-  
>name;  
}
```

In raw SQL, JOIN instead of looping:

```
SELECT orders.*, users.name  
FROM orders  
JOIN users ON orders.user_id = users.id;
```

Slow Query Log

```
SET GLOBAL slow_query_log = 'ON';  
SET GLOBAL long_query_time = 1; -- log queries over 1 second
```

```
# Summarise: sort by time, top 10  
mysqldumpslow -s t -t 10 /var/log/mysql/mysql-slow.log
```

Other Common Bottlenecks

SELECT *

Fetches columns you don't need. Select only what you use.

```
-- GOOD
SELECT id, name, email
FROM users WHERE id = 1;
```

LEADING WILDCARD

`LIKE '%x%'` can't use an index. Use a FULLTEXT index for search.

```
ALTER TABLE products
  ADD FULLTEXT(name);
SELECT * FROM products
WHERE MATCH(name)
  AGAINST('widget');
```

Missing WHERE on large tables — always filter early. Transactions — wrap related writes so they commit together and stay consistent:

```
START TRANSACTION;
INSERT INTO orders (user_id, total) VALUES (1, 99.99);
INSERT INTO order_items (order_id, product_id) VALUES
(LAST_INSERT_ID(), 5);
COMMIT;
```

5 · REST API Design

CORE PRINCIPLES

- Resources are nouns, not verbs
- Use HTTP methods to express intent
- Stateless — each request carries everything it needs

METHODS & ROUTES

GET	/users	list
GET	/users/42	fetch
one		
POST	/users	create
PUT	/users/42	replace
PATCH	/users/42	partial
update		
DELETE	/users/42	delete

STATUS CODES

200 OK	success
201 Created	POST
created	
204 No Content	DELETE ok
400 Bad Request	invalid
data	
401 Unauthorized	not
auth'd	
403 Forbidden	not
allowed	
404 Not Found	
422 Unprocessable	
validation	
429 Too Many	rate
limited	
500 Server Error	

Versioning

Always version so you can ship breaking changes without breaking clients: `/api/v1/users`, `/api/v2/users`.

Clean endpoint (Laravel)

```
public function show(int $id): JsonResponse
{
    $user = $this->userRepository->findById($id);

    if (!$user) {
        return response()->json(['error' => 'User not found'], 404);
    }

    return response()->json($user, 200);
}
```

6 · Docker & Containerisation

WHY IT MATTERS

BaseKit use Docker and Kubernetes. You don't need to be a DevOps expert — know the basics and the vocabulary.

KEY CONCEPTS

- Image — a blueprint (like a class)
- Container — a running instance (like an object)
- Dockerfile — build instructions
- docker-compose — run many containers together

ESSENTIAL COMMANDS

```
docker ps #
running
docker ps -a #
all
docker logs -f myapp #
tail logs
docker exec -it myapp bash
docker build -t
app:latest .
docker run -d -p 8080:80
nginx
```

Typical PHP Dockerfile

```
FROM php:8.2-fpm
RUN docker-php-ext-install pdo pdo_mysql
WORKDIR /var/www
COPY . .
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
RUN composer install --no-dev --optimize-autoloader
EXPOSE 9000
CMD ["php-fpm"]
```

docker-compose

```
services:
  app:
    build: .
    ports: ["8000:8000"]
    depends_on: [db]
    environment:
      DB_HOST: db
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: myapp
    volumes:
      - db_data:/var/lib/mysql
volumes:
  db_data:
```

```
docker-compose up -d      # start all
docker-compose down        # stop all
docker-compose logs -f     # tail all
```

KUBERNETES — HIGH LEVEL

Orchestrates containers at scale. Know the four nouns:

- Pod — smallest unit, one or more containers
- Deployment — rolling updates and replicas
- Service — exposes pods to network traffic
- Ingress — routes external traffic to services

7 · Linux & Mac Command Line

The commands that come up day-to-day as a backend dev. Mac and Linux both use bash/zsh.

NAVIGATE & FILES

```
pwd
ls -la
cd /var/www/html
tail -f logfile.log
grep -r "findme" ./
find . -name "*.php"
```

PROCESSES

```
ps aux | grep php
kill 1234
kill -9 1234      # force
top
htop
```

PERMISSIONS

```
chmod 755 script.sh
chmod +x script.sh
chown www-data:www-data /
var/www
sudo command
```

NETWORKING

```
curl -I https://example.
com
wget https://host/file.zip
netstat -tulpn
lsof -i :8080
```


SSH & COPY

```
ssh user@host
ssh -i ~/.ssh/key
user@host
scp file.txt user@host:/
path
```

SHORTCUTS

```
Ctrl+C    kill process
Ctrl+R    search history
!!        repeat last
command
!$        last arg of prev
cmd
```

PHP & COMPOSER

```
php -v          # version
php -m          # modules
php -r "echo phpversion();"
php artisan migrate # Laravel
composer install
composer dump-autoload
```

8 · Git Workflow

EVERYDAY

```
git status
git diff
git add .
git commit -m "message"
git push origin feature/x
git pull origin main
git log --oneline --graph
```

BRANCHING

```
git checkout -b feature/new
git checkout main
git merge feature/new
git rebase main
git branch -d feature/new
```

FIXING MISTAKES

```
git stash / git stash pop
git reset HEAD~1 #
keep changes
git reset --hard HEAD~1 #
DISCARD
git revert abc1234 #
safe on shared
```

COMMIT MESSAGE FORMAT

```
feat: add auth endpoint
fix: null pointer in
processor
refactor: extract payment
service
chore: bump composer deps
```

9 · OOP Concepts

The four pillars

ENCAPSULATION

Hide internal state; expose only what's needed.

```
class BankAccount {
    private float $balance
    = 0;
    public function
    deposit(float $a): void {
        if ($a <= 0) throw
        new
        InvalidArgumentException();
        $this->balance +=
        $a;
    }
}
```

ABSTRACTION

Expose what something does, not how.

```
interface Cache {
    public function
    get(string $k): mixed;
    public function
    set(string $k, mixed $v,
    int $ttl): void;
}
// Redis, Memcached, file
– same interface
```

INHERITANCE

Child classes extend parent behaviour.

```
abstract class
Notification {
    abstract public
    function send(string $m):
    void;
    public function
    log(string $m): void { /
    * ... */ }
}
class EmailNotification
extends Notification {
    public function
    send(string $m): void { /
    * ... */ }
}
```

POLYMORPHISM

Same call, different behaviour per class.

```
foreach ($notifications as
$n) {
    $n->send("Hello!");
    // each type sends
    differently
}
```

Composition over Inheritance

WHY IT MATTERS

BaseKit explicitly prefer this. Instead of inheriting behaviour, inject it — flatter hierarchies, easier testing.

✗ INHERITANCE — RIGID

```
class LoggingOrderService
  extends OrderService {
  public function
  create(): void {
    $this->log("Creating order");
    parent::create();
  }
}
```

✓ COMPOSITION — FLEXIBLE

```
class OrderService {
  public function
  __construct(
    private
    OrderRepository $repo,
    private
    LoggerInterface $logger,
  ) {}
  public function
  create(): void {
    $this->logger->info("Creating order");
    $this->repo->save();
  }
}
```

Say this: “I prefer composing behaviour through injected dependencies over deep inheritance hierarchies.”

10 · Security Essentials

SQL Injection

Never interpolate user input into SQL. Use prepared statements.

✗ VULNERABLE

```
$id = $_GET['id']; // 1 OR 1=1
$db->query(
    "SELECT * FROM users
    WHERE id = $id"
);
```

✓ SAFE

```
$stmt = $db->prepare(
    "SELECT * FROM users
    WHERE id = ?"
);
$stmt->execute([$id]);
```

XSS — escape output

```
// SAFE
echo "<p>Hello " . htmlspecialchars($_GET['name'], ENT_QUOTES,
'UTF-8') . "</p>";
```

CSRF

Use tokens to prove a form came from your own site. Laravel does this automatically with `@csrf`.

PASSWORD HASHING

Never plain text, MD5 or SHA1.

```
$hash = password_hash($pw,
PASSWORD_BCRYPT);
if (password_verify($pw,
$hash)) { /* ok */ }
```

SECRETS

Never commit secrets. Keep them in `.env`, and put `.env` in `.gitignore`.

11 · General Best Practices

CODE REVIEW MINDSET

- Readable in 6 months?
- Is it tested?
- Edge cases handled?
- Is there a simpler way?

THREE ACRONYMS

- DRY — extract repeated logic
- YAGNI — don't build for needs that don't exist
- KISS — simplest thing that works; complexity must be earned

Testing basics (PHPUnit)

```
class OrderServiceTest extends TestCase {
    public function test_creates_order_successfully(): void {
        $mockRepo = $this->createMock(OrderRepository::class);
        $mockRepo->expects($this->once())->method('save');

        $service = new OrderService($mockRepo);
        $service->create(['user_id' => 1, 'total' => 50.00]);
    }
}
```

If something is hard to test, it's usually a sign the design needs revisiting.

Part Two · Deeper Technical Topics

The first half is the breadth refresher. This half goes deeper into the things a technical round tends to probe — language internals, data integrity, caching, async, and how a PHP app actually runs and scales.

LANGUAGE DEPTH

- 12. Type System & Gotchas
- 13. Arrays & Higher-Order Fns
- 14. Traits / Abstract / Interface
- 15. Generators & Closures
- 16. Errors & Exceptions

ARCHITECTURE & DATA

- 17. Design Patterns
- 18. PSR & Composer
- 19. ACID & Isolation
- 20. Caching Strategies
- 21. Queues & Background Jobs

RUNTIME & SCALE

- 22. Auth & HTTP
- 23. Request Lifecycle / FPM
- 24. Testing Deeper
- 25. Big-O & Coding
- 26. Scaling & Multi-Tenancy

12 · Type System & Gotchas

WHY IT MATTERS

“What does this print?” is a classic technical-round opener. Type juggling and loose comparison catch out anyone who hasn’t looked closely at PHP’s rules — and PHP 8 quietly changed several of them.

== vs === (loose vs strict)

``==`` coerces types before comparing; ``===`` checks value and type. Default to ``===`` unless you want coercion.

```
var_dump(0 == "a");      // PHP 7: true (!)   PHP 8: false
var_dump("1" == 1);     // true  - numeric string
var_dump("abc" == 0);    // PHP 7: true      PHP 8: false
var_dump(null == false); // true
var_dump([] == false);   // true
var_dump(0 === "0");     // false - different types
```

The PHP 8 fix: comparing a number to a non-numeric string now casts the number to string, so ``0 == "abc"`` is finally ``false``. Knowing this one signals you’ve actually used PHP 8.

Handy operators

```
$name = $data['name'] ?? 'guest'; // null coalescing - no warning if
unset
$config['x'] ??= 'default';        // assign only if null/unset
$cmp = $a <=> $b;                  // spaceship: -1, 0, or 1
declare(strict_types=1);           // top of file - enforce type
declarations
```

Say this: “I put ``declare(strict_types=1)`` at the top of every file and use ``===`` by default — type juggling should be a deliberate choice, not an accident.”

13 · Arrays & Higher-Order Functions

PHP arrays are ordered maps — they double as lists, dictionaries, stacks and queues. Knowing the functional trio cold reads as fluency.

```
$nums = [1, 2, 3, 4];

$doubled = array_map(fn($n) => $n * 2, $nums);           // [2, 4, 6, 8]
$evens   = array_filter($nums, fn($n) => $n % 2 === 0); // [1 => 2, 3 => 4]
$sum      = array_reduce($nums, fn($c, $n) => $c + $n, 0); // 10
```

GOTCHA — KEYS

`array_filter` preserves keys, leaving gaps. Reindex with `array_values()` before sending as a JSON array.

```
$evens = array_values(
    array_filter($nums,
        $isEven)
);
```

WORTH KNOWING

- `array_column()` — pluck a field
- `array_combine()` — keys + values
- `usort()` — custom sort with `<=>`
- `[...$a, ...$b]` — spread/merge

`array_map` vs `foreach`: `array_map` is declarative and returns a new array (no mutation). Reach for a `foreach` when you need the index, early exit, or side effects.

13 · Arrays on Real Data

The trio shines on rows from the database. Say you've fetched a list of orders:

```
$orders = [
  ['id' => 1, 'user' => 'Ann', 'total' => 30, 'paid' => true],
  ['id' => 2, 'user' => 'Bob', 'total' => 80, 'paid' => false],
  ['id' => 3, 'user' => 'Ann', 'total' => 50, 'paid' => true],
];
```

PLUCK A COLUMN

```
$ids =
array_column($orders,
'id');
// [1, 2, 3]

// key it by id:
$byId = array_column(
  $orders, null, 'id'
);
```

SUM THE PAID ONES

```
$revenue = array_sum(
  array_column(
    array_filter(
      $orders,
      fn($o) =>
        $o['paid']
    ),
    'total'
  )
); // 80
```

Grouping (the one people forget)

There's no `array\_group` — you build it with a `foreach`. Worth knowing by heart:

```
$byUser = [];
foreach ($orders as $o) {
  $byUser[$o['user']][] = $o; // append into a bucket keyed by
  user
}
// 'Ann' => [order 1, order 3], 'Bob' => [order 2]
```

Sorting by a field uses the spaceship operator inside `usort`:

```
usort($orders, fn($a, $b) => $b['total'] <=> $a['total']); // highest
total first
```

14 · Traits, Abstract & Interface

WHY IT MATTERS

“When would you use a trait over an abstract class?” tests whether you understand composition, contracts and PHP’s single-inheritance model.

INTERFACE

A contract — no implementation. A class can implement many. “Can do X.”

ABSTRACT CLASS

Partial implementation + shared state. Single inheritance, can’t be instantiated. “Is a kind of X.”

TRAIT

Horizontal code reuse — copy-pasted in at compile time. Use for shared behaviour across unrelated classes.

```
interface Notifier { public function send(string $msg): void; }

abstract class BaseRepository {
    abstract protected function table(): string;    // subclass must
    define
    public function all(): array { /* uses $this->table() */ }
}

trait Timestamps {
    public function touch(): void { $this->updatedAt = new
    DateTime(); }
}

class User { use Timestamps; }
```

self vs static (late static binding)

```
class Model {
    public static function create(): static {
        return new static();    // resolves to the CALLED class, not
        Model
    }
}

// User::create() returns a User, not a Model
```

15 · Generators & Closures

Generators — lazy iteration

`yield` produces values one at a time without building the whole array in memory. The interview angle: processing a huge file or result set without running out of memory.

```
function readLines(string $file): Generator {
    $h = fopen($file, 'r');
    while (($line = fgets($h)) !== false) {
        yield trim($line);    // one line in memory at a time
    }
    fclose($h);
}

foreach (readLines('huge.log') as $line) {
    // constant memory, even for a 10GB file
}
```

Closures & captured scope

```
$tax = 0.2;

// Arrow fn – auto-captures by value
$withTax = fn(float $net) => $net * (1 + $tax);

// Classic closure – explicit capture; & captures by reference
$total = 0;
$add = function (int $n) use (&$total) { $total += $n; };
```

Key distinction: arrow functions (`fn`) capture outer variables automatically by value; classic `function () use (...)` closures capture explicitly, and `&` makes the capture by reference.

16 · Errors & Exceptions

Everything throwable implements `Throwable`, which splits into `Error` (engine-level: `TypeError`, `DivisionByZeroError`) and `Exception` (application-level, the ones you throw and catch).

```
try {
    $receipt = $payments->charge($order);
} catch (InsufficientFundsException $e) {
    // catch the SPECIFIC case first
    return $this->declineGracefully($e);
} catch (PaymentException | NetworkException $e) {    // PHP 8 union
    catch
        $logger->error('Charge failed', ['order' => $order->id, 'e' =>
            $e]);
        throw new OrderFailedException('Could not place order', previous:
            $e);
} finally {
    $lock->release();    // always runs – cleanup
}
```

CUSTOM EXCEPTIONS

```
class
    InsufficientFundsException
        extends
            DomainException {}
```

Extend `DomainException`, `RuntimeException`, etc. so callers can catch by type.

DON'T DO THIS

- Swallow exceptions with an empty `catch`
- Catch `Exception` everywhere
- Use exceptions for normal control flow
- Lose the original via `previous`

17 · Design Patterns

WHY IT MATTERS

BaseKit value clean architecture. You don't need to recite the Gang of Four — but naming the right pattern for a problem, and knowing one anti-pattern, lands well.

Pattern	Use it to...
Repository	Hide persistence behind an interface — swap MySQL for a fake in tests.
Factory	Centralise complex object creation; return an interface, not a concrete.
Strategy	Swap interchangeable behaviours (e.g. payment gateways) without if-ladders.
Observer	Let many listeners react to an event (e.g. 'order placed') without coupling.
Adapter	Wrap a third-party API in your own interface so vendor changes stay contained.
Decorator	Add behaviour (logging, caching) by wrapping, not subclassing.

STRATEGY — THE GO-TO

```
interface ShippingRate {
    public function
    cost(Order $o): float;
}
class Standard implements
ShippingRate { /* ... */ }
class Express implements
ShippingRate { /* ... */ }

// Inject the chosen
strategy — no switch
statement
$checkout = new
Checkout($express);
```

SINGLETON — ⚠️ ANTI-PATTERN

Hidden global state, hard to test, hides dependencies. In modern PHP the DI container manages a single shared instance for you — without the downsides. Mention this trade-off if asked.

17 · Patterns in Code

Repository — hide the database

Callers depend on an interface, not on MySQL. Swap a fake in tests instantly.

```
interface UserRepository {
    public function find(int $id): ?User;
    public function save(User $user): void;
}

class MySqlUserRepository implements UserRepository { /* real SQL */ }
class InMemoryUserRepository implements UserRepository { /* array, for tests */ }

// The service neither knows nor cares which one it got
class Onboarding {
    public function __construct(private UserRepository $users) {}
}
```

Factory — centralise tricky creation

```
class GatewayFactory {
    public function make(string $type): PaymentGateway {
        return match ($type) {
            'stripe' => new StripeGateway(config('stripe.key')),
            'paypal' => new PaypalGateway(config('paypal.token')),
            default => throw new InvalidArgumentException("Unknown:
$type"),
        };
    }
}
```

Observer — react without coupling

The order doesn't know who's listening; listeners subscribe to the event.

```
$dispatcher->listen(OrderPlaced::class, fn($e) => $mailer->sendReceipt($e->order));
$dispatcher->listen(OrderPlaced::class, fn($e) => $warehouse->reserve($e->order));

// Later, the order just announces what happened:
$dispatcher->dispatch(new OrderPlaced($order));
```

18 · PSR Standards & Composer

PSRs are the shared conventions that let PHP packages interoperate. Know what the common ones cover.

THE ONES TO NAME

- PSR-4 — autoloading (namespace → path)
- PSR-12 — coding style
- PSR-3 — logger interface
- PSR-7 / 15 — HTTP messages & middleware
- PSR-11 — container interface

SEMVER

`^7.2` → any `7.x` (≥ 7.2)
`~7.2` → `7.2.x` only

`MAJOR.MINOR.PATCH` — major = breaking, minor = features, patch = fixes.

composer.json

```
{
  "require": {
    "php": "^8.2",
    "guzzlehttp/guzzle": "^7.0"
  },
  "autoload": {
    "psr-4": { "App\\": "src/" }
  }
}
```

```
composer require monolog/monolog          # add a dependency
composer install --no-dev --optimize-autoloader # production install
composer dump-autoload                     # rebuild the autoloader
```


19 · ACID & Isolation Levels

WHY IT MATTERS

Money, bookings, anything that must not half-happen — this is where backend rigour shows. Expect questions on transactions, locking and race conditions.

ACID

- Atomic — all or nothing
- Consistent — valid state → valid state
- Isolated — concurrent txns don't interfere
- Durable — committed data survives a crash

SAFE WRITE

```
$db->beginTransaction();
try {
    $db->debit($from,
100);
    $db->credit($to, 100);
    $db->commit();
} catch (Throwable $e) {
    $db->rollBack();
    throw $e;
}
```

Isolation levels (weakest → strongest)

Level	Allows	Notes
READ UNCOMMITTED	Dirty reads	Rarely used
READ COMMITTED	Non-repeatable reads	Postgres default
REPEATABLE READ	Phantom reads*	MySQL/InnoDB default
SERIALIZABLE	Nothing — fully isolated	Safest, slowest

Use `SELECT ... FOR UPDATE` to lock rows you're about to change and avoid lost updates. InnoDB's gap locking largely prevents phantoms even at REPEATABLE READ.

19 · A Race Condition in Action

WHY IT MATTERS

This is the classic “two requests at the same time” question. If you can describe the bug and the fix, you sound like someone who’s run production.

Imagine the last ticket to a gig. Two requests check stock at the same instant:

✗ THE BUG — LOST UPDATE

```
// Both requests read
stock = 1
$stock = $db->query(
    "SELECT stock FROM
    events WHERE id=7"
);
if ($stock > 0) {
    // both pass the
    check...
    $db->query(
        "UPDATE events
        SET stock = stock -
        1
        WHERE id = 7"
    );
}
// Sold 2 tickets. Stock
is now -1.
```

✓ THE FIX — LOCK THE ROW

```
$db->beginTransaction();
// FOR UPDATE makes the
2nd request
// WAIT until the 1st
commits
$stock = $db->query(
    "SELECT stock FROM
    events
    WHERE id = 7 FOR
    UPDATE"
);
if ($stock > 0) {
    $db->query(
        "UPDATE events
        SET stock = stock -
        1
        WHERE id = 7"
    );
}
$db->commit();
```

Even simpler: push the check into the write itself — `UPDATE events SET stock = stock - 1 WHERE id = 7 AND stock > 0` — then check the affected-row count. The database does the locking for you, and stock can never go negative.

20 · Caching Strategies

“There are only two hard things in computer science: cache invalidation and naming things.” Know the layers and the cache-aside pattern.

THE LAYERS

- OPcache — compiled PHP bytecode (turn on in prod)
- App cache — Redis / Memcached for data
- HTTP cache — `Cache-Control`, ETags, CDN
- Query cache — materialised/computed results

CACHE-ASIDE

```
function getUser(int $id):  
array {  
    $key = "user:$id";  
    if ($hit = $redis->  
>get($key)) {  
        return  
        json_decode($hit, true);  
    }  
    $user = $db->  
>find($id);  
    $redis->setex($key,  
    3600, json_encode($user));  
    return $user;  
}
```

Invalidation: set a sensible TTL, and bust the key on write (`\$redis->del("user:\$id")` after an update). Write-through updates cache and DB together; cache-aside (above) is the common default. Always have a plan for a cold cache — a thundering herd of misses can take the DB down.

21 · Queues & Background Jobs

WHY IT MATTERS

Anything slow — sending email, resizing images, calling a third-party API — shouldn't block the HTTP response. Knowing why and how to defer work is a core backend signal.

A producer pushes a job onto a broker (Redis, RabbitMQ, SQS); a separate worker process pulls and runs it asynchronously.

```
// Producer — returns immediately, user isn't kept waiting
SendWelcomeEmail::dispatch($user)->onQueue('emails');

// Worker (run as a long-lived process / container)
//  php artisan queue:work --queue=emails --tries=3
```

GET RIGHT

- Idempotent jobs — safe to run twice
- Retries with backoff
- Dead-letter queue for poison jobs
- Pass IDs, not whole objects

WHY IT MATTERS

Faster responses, resilience to spikes (the queue absorbs load), and failures that retry instead of losing data. Workers scale independently of web servers.

22 · API Auth & HTTP

SESSION (STATEFUL)

Server stores the session; client holds a cookie. Easy to revoke; needs shared session storage (Redis) when scaled.

JWT (STATELESS)

Signed token holds the claims: `header.payload.signature`. Nothing stored server-side — but hard to revoke before expiry. Keep them short-lived + use refresh tokens.

```
Authorization: Bearer  
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiJ9.<signature>
```

Things they may poke at

- OAuth2 — delegated access; authentication (who you are) vs authorization (what you can do).
- CORS — the browser's pre-flight `OPTIONS` check; a server header, not a security boundary.
- Idempotency — `GET`, `PUT`, `DELETE` are idempotent; `POST` is not. Matters for safe retries.
- Always HTTPS, never log tokens, hash anything secret at rest.

23 · Request Lifecycle & PHP-FPM

WHY IT MATTERS

A favourite senior question: “What actually happens when a request hits a PHP app?” The shared-nothing model explains a lot of PHP’s behaviour.

THE PATH OF A REQUEST

nginx receives the request → forwards to a PHP-FPM pool over FastCGI → an FPM worker bootstraps the app, handles the request, sends the response → the worker is reset and reused for the next request.

SHARED-NOTHING

Every request starts with a clean slate — no variables persist between requests (unlike Node). So global in-memory state is useless across requests.

CONSEQUENCES

- Sessions/cache live in Redis or the DB, not memory
- OPcache keeps compiled bytecode hot
- Statelessness makes horizontal scaling trivial

Say this: “PHP is shared-nothing — each request is isolated, which is why state goes in Redis or the database and why we can scale just by adding more FPM workers behind the load balancer.”

24 · Testing, Deeper

THE PYRAMID

Many unit tests (fast, isolated), fewer integration tests (real DB), fewest end-to-end (slow, brittle). Optimise the bulk for speed.

AAA + TDD

Arrange, Act, Assert per test. TDD = red → green → refactor: write a failing test, make it pass, then clean up.

Test doubles & data providers

Stub returns canned data; mock also asserts it was called; fake is a working lightweight implementation (e.g. in-memory repo); spy records calls. Data providers run one test over many cases:

```
#[DataProvider('vatCases')]
public function test_vat_is_added(float $net, float $expected): void {
    $this->assertSame($expected, addVat($net));
}

public static function vatCases(): array {
    return [
        'standard' => [100.0, 120.0],
        'zero'     => [0.0, 0.0],
    ];
}
```

Coverage is a guide, not a goal — 100% coverage of trivial getters proves little; one good test of the money path proves a lot.

25 · Big-O Notation

WHY IT MATTERS

Big-O is just a way of answering one question: as the input gets bigger, how fast does the work grow? It's not about seconds — it's about shape. Interviewers use it to check you can reason about cost before you write code.

The mental model. n is the size of the input (number of items). Big-O describes the number of steps relative to n . We ignore constants and only keep the dominant term — $O(2n + 5)$ is just $O(n)$, because when n is huge the rest stops mattering.

Why it matters — the numbers

This table is the whole point. It shows roughly how many steps each class takes as the input grows. Watch what $O(n^2)$ does.

Input size n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
10	1	3	10	33	100
100	1	7	100	664	10,000
1,000	1	10	1,000	9,966	1,000,000
1,000,000	1	20	1 million	~20 million	1 trillion ☠

$O(1)$ doesn't care how big the input is. $O(\log n)$ barely grows — a million items takes ~20 steps. $O(n^2)$ explodes — a million items is a trillion steps, i.e. it will never finish. That's the difference.

25 · A Code Example for Each

Same idea, one function per class. The thing to notice is the loop shape.

$O(1)$ · CONSTANT

No loop — jump straight to the answer.
Same cost for 10 or 10 million.

```
$first =
$users[0];          // index
$exists =
isset($map[$key]); // hash
```

$O(n)$ · LINEAR

One loop over the input — touch each item once.

```
$total = 0;
foreach ($orders as $o) {
    $total += $o->amount;
}
```

$O(\log n)$ · logarithmic — halve the problem each step

Binary search on a sorted array. Each step throws away half of what's left, so a million items takes only ~20 steps.

```
function binarySearch(array $sorted, int $target): int {
    $lo = 0; $hi = count($sorted) - 1;
    while ($lo <= $hi) {
        $mid = intdiv($lo + $hi, 2);
        if ($sorted[$mid] === $target) return $mid; // found
        if ($sorted[$mid] < $target) $lo = $mid + 1; // discard left
        else $hi = $mid - 1; // discard right half
    }
    return -1;
}
```

$O(n \log n)$ · the cost of a good sort

```
sort($nums); // ~ $O(n \log n)$ 
usort($users, fn($a, $b) => $a->age <=> $b->age);
```

25 · The $O(n^2)$ Trap & The Fix

$O(n^2)$ · quadratic — a loop inside a loop over the same data. This is the one to watch for: it looks innocent and dies on real data.

✗ $O(n^2)$ — NESTED LOOPS

Compares every item against every other — 1,000 items = 1,000,000 comparisons.

```
// Has this array got
// duplicates?
foreach ($items as $i =>
$a) {
    foreach ($items as $j
=> $b) {
        if ($i !== $j &&
$a === $b) {
            return true;
        }
    }
}
```

✓ $O(n)$ — ONE PASS + HASH

Remember what you've seen in a hash map — 1,000 items = 1,000 steps.

```
$seen = [];
foreach ($items as $a) {
    if (isset($seen[$a]))
    {
        return true;
    }
    $seen[$a] = true;
}
return false;
```

HOW TO SPOT IT AT A GLANCE

- No loop, direct access → $O(1)$
- Halving the search space each step → $O(\log n)$
- One loop over the input → $O(n)$
- A sort, or divide-and-conquer → $O(n \log n)$
- A loop inside a loop over the same data → $O(n^2)$ ⚠

The interview move

Trading nested loops for a hash map — $O(n^2) \rightarrow O(n)$ — is the single most common optimisation they look for. The classic two-sum shows it: instead of checking every pair, remember each number and look up its complement.

```
function twoSum(array $nums, int $target): ?array {
    $seen = [];
    foreach ($nums as $i => $n) {
        if (isset($seen[$target - $n])) {           // 0(1) lookup, not a
second loop
            return [$seen[$target - $n], $i];
        }
        $seen[$n] = $i;
    }
    return null;
}
```

Say this: “The brute force is $O(n^2)$ with nested loops. I can trade memory for speed — a hash map of what I’ve seen gets it to $O(n)$ in a single pass.” That sentence is what they’re listening for.

26 · Scaling & Multi-Tenancy

WHY IT MATTERS

BaseKit is a white-label website builder sold through partners — a multi-tenant SaaS serving many customers from shared infrastructure. Showing you understand their shape of problem is a real edge.

SCALING BASICS

- Vertical — bigger box (limited)
- Horizontal — more boxes behind a load balancer (the PHP way)
- Stateless app servers make this trivial
- Read replicas for read-heavy load
- CDN for static assets

MULTI-TENANCY MODELS

- Shared DB + `tenant\_id` — simplest, must scope every query
- Schema per tenant — stronger isolation
- DB per tenant — strongest, heaviest to run

The trap to mention: in a shared-database model, a single query that forgets its `tenant\_id` scope leaks one customer's data to another. A global query scope (or row-level security) makes that the default, not something every developer must remember.

Part Three · Running PHP in Production

Parts One and Two cover the language and the patterns. This part is the operational half — the questions a senior backend round leans on hardest: how do you make a database fast, find out why a page is slow, and run the same code reliably on a server that you only ever ran on your laptop.

DATA AT SPEED

- 27. Database Optimisation, Deeper
- 28. Diagnosing Slow Page Loads
- 29. Connections & Big Result Sets

THE RUNTIME

- 30. PHP Versions & Performance
- 31. PHP-FPM & OPcache Tuning

ENVIRONMENTS

- 32. Local vs Staging vs Production
- 33. Observability & Debugging Prod

27 · Database Optimisation, Deeper

WHY IT MATTERS

Section 4 was the refresher — EXPLAIN, indexes, N+1. This goes a level down: how indexes actually work, when an index won't be used, and the schema-level moves that fix slow reads. This is where a senior answer separates from a junior one.

How an index actually works

InnoDB stores the table itself as a B-tree keyed on the primary key — the clustered index. A secondary index is a second B-tree whose leaves hold the PK, so a lookup by a secondary index is two hops: find the PK in the index, then fetch the row from the clustered index.

WHY THIS MATTERS

- The PK sits inside every secondary index — keep it small (a `BIGINT`, not a random UUID string).
- A query needing columns not in the index does a second lookup per row. A covering index avoids that.
- Random PKs (UUIDv4) cause page splits and fragmentation on insert.

COVERING INDEX

If the index holds every column the query needs, MySQL answers from the index alone — no row fetch. EXPLAIN shows `Using index`.

```
CREATE INDEX idx_cover
  ON orders (user_id,
    status, total);
-- SELECT total FROM
orders
-- WHERE user_id=1 AND
status='paid'
-- answered from the index
alone
```

When an index won't be used

Knowing these saves hours of head-scratching:

```
-- x function on the column → index ignored
WHERE YEAR(created_at) = 2024;
-- ✓ rewrite as a range so the index works
WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01';

-- x leading wildcard → no index
WHERE name LIKE '%smith';
-- x type mismatch → silent cast, index skipped
WHERE phone = 0791123456; -- phone is VARCHAR; quote it
-- x low selectivity → optimiser prefers a scan
WHERE is_active = 1; -- if 95% of rows are active
```

27 · Schema & Query Moves

Composite index column order

Order columns equality first, then range, then sort. The optimiser stops using the index at the first range condition, so put ranges last.

```
-- Query: WHERE tenant_id = ? AND status = ? AND created_at > ? ORDER
BY created_at
CREATE INDEX idx ON orders (tenant_id, status, created_at);
--                      equality    equality  range + sort
-- Satisfies the filter AND the sort – no 'Using filesort'.
```

Reading EXPLAIN like a senior

Column	Want to see	Red flag
type	const, eq_ref, ref, range	ALL (full scan)
key	the index you expect	NULL
rows	small vs table size	millions
filtered	high %	low % = scan then discard
Extra	Using index	Using filesort / temporary

Schema-level fixes for slow reads

DENORMALISE A COUNTER

Don't `COUNT(\*)` a million rows on every page load. Keep a `comment\_count` column, updated in the same transaction as the write.

RIGHT-SIZE TYPES

`INT` not `BIGINT` where it fits;
 `VARCHAR(50)` not `(255)`;
 `TIMESTAMP` (4 bytes) over
 `DATETIME` (8). Smaller rows → more
 per page → fewer disk reads.

Partitioning & archiving: past tens of millions of rows, partition by date or move cold rows to an archive table so the hot working set stays small. Read replicas take read load off the primary; route reads to a replica, writes to the primary.

Say this: “Before adding an index I run EXPLAIN, check selectivity, and order composite columns equality-first. I’ll reach for a covering index or a denormalised count before I reach for more hardware.”

28 · Diagnosing Slow Page Loads

WHY IT MATTERS

“A page is slow — how do you find out why?” is one of the most revealing questions they ask. The weak answer guesses and starts optimising. The strong answer measures first, isolates the layer, then fixes the biggest cost.

THE METHOD — MEASURE, DON'T GUESS

1. Reproduce and time it end-to-end (browser dev tools → Network).
2. Split the time: server (TTFB) or front end (assets, JS)?
3. On the server, profile to find the hot spot — DB? external API? CPU?
4. Fix the biggest cost first. Re-measure. Repeat.

WHERE TIME ACTUALLY GOES

- Database — too many queries ($N+1$), or one slow query
- External APIs — a synchronous call to a third party
- No caching — recomputing the same thing every request
- PHP CPU — a tight loop, $O(n^2)$, huge (de)serialization
- Front end — unoptimised images, render-blocking JS

FIRST QUESTIONS

- How many queries does this page run? (often the answer)
- Slow for everyone, or one tenant with more data?
- Slow always, or only under load?
- Did it just start, or always been slow?

28 · The Tools

Name these and you sound like someone who's debugged real performance problems.

PROFILERS

- Xdebug profiler — function-level timings (view in KCacheGrind)
- Blackfire / Tideways — production-grade call graphs; wall time vs CPU vs I/O
- ``microtime(true)`` — quick manual timing of a suspect block

APM & QUERY VISIBILITY

- New Relic / Datadog APM — transaction traces in prod
- Laravel Debugbar / Clockwork — query count + timings per request
- MySQL slow query log + ``EXPLAIN`` on the offenders

Quick-and-dirty timing

```
$start = microtime(true);
$result = $expensiveThing();
$ms = (microtime(true) - $start) * 1000;
$logger->info('expensiveThing', ['ms' => round($ms, 1)]);

// Count queries in dev (Laravel)
DB::enableQueryLog();
// ... run the page ...
logger()->info('queries', ['n' => count(DB::getQueryLog())]);
```

Say this: “I profile before I change anything — Blackfire or the slow query log to find the actual hot spot, rather than optimising the thing I assume is slow. Nine times out of ten it's query count, not raw CPU.”

29 · Connections & Big Result Sets

WHY IT MATTERS

Two things quietly kill PHP apps at scale: opening too many database connections, and loading too many rows into memory at once. Both have clean fixes.

Connection handling — the PHP reality

PHP is shared-nothing: each FPM worker opens its own DB connection, and there's no built-in connection pool like a long-running Node or Java app. So connections scale with FPM workers — and that bites.

THE MATH THAT BITES

50 FPM workers × 3 app servers = 150 connections, even idle. MySQL's `max\_connections` default is 151. Add a traffic spike and you hit "Too many connections" — an outage.

THE FIX

Put a pooler in front — ProxySQL (MySQL) or PgBouncer (Postgres). It multiplexes many app connections onto a few DB ones. Size the FPM pool deliberately; don't just crank it up.

Persistent connections (`PDO::ATTR\_PERSISTENT`) reuse a connection across requests in the same worker — fewer handshakes, but they hold a slot and can leak transaction/session state. Prefer an external pooler.

Don't load a million rows

`Model::all()` on a big table pulls every row into memory, then PHP hits `memory\_limit` and dies. Stream instead:

```
// ✗ loads everything into memory
foreach (Order::all() as $o) { export($o); }

// ✓ chunk — N rows at a time, constant memory
Order::chunkById(1000, function ($orders) {
    foreach ($orders as $o) { export($o); }
});

// ✓ or a lazy cursor (a generator under the hood)
foreach (Order::lazy() as $o) { export($o); }
```

Same idea in raw PHP: a generator (`yield`) or an unbuffered query streams rows instead of buffering the whole set. And always paginate list endpoints — `LIMIT`/`OFFSET`, or keyset pagination for deep pages.

30 · PHP Versions & Performance

WHY IT MATTERS

“Why upgrade PHP?” has a great one-line answer: it’s free performance. Each modern release got materially faster while your code stayed the same. Knowing the timeline and what changed signals you keep stacks current.

Version	Released	Why it matters
7.0	2015	The big one — ~2x faster than 5.6, half the memory (new engine).
7.4	2019	Preloading, typed properties, arrow fns. End of the 7.x line (EOL).
8.0	2020	JIT, named args, match, union types, attributes.
8.1	2021	Enums, readonly props, fibers, first-class callables.
8.2	2022	readonly classes, DNF types, no more dynamic properties.
8.3 / 8.4	2023 / 24	Typed constants, #[Override], property hooks (8.4).

THE JIT — BE HONEST

PHP 8’s JIT compiles hot paths to machine code. It helps CPU-bound work (image processing, maths) a lot — but most web apps are I/O-bound (waiting on DB and network), so JIT’s real-world web gain is modest. Saying that shows you understand it, not just that it exists.

STAYING CURRENT

- Each version: ~2 yrs active support + 1 yr security
- Running EOL PHP = unpatched security holes
- Upgrades are mostly cheap; Rector automates much of the migration
- Test against the new version in CI before bumping prod

Say this: “Upgrading PHP is some of the cheapest performance you can buy — 7.0 roughly doubled throughput. I’d keep us on a supported version, use Rector to automate the migration, and gate it behind CI.”

31 · PHP-FPM & OPcache Tuning

WHY IT MATTERS

Section 23 covered the request lifecycle. This is the operational follow-up: the handful of settings that decide how much traffic one box can take. You won't be asked to memorise them — knowing what they do is the signal.

OPcache — turn it on, always

Without OPcache, PHP recompiles every `.php` file on every request. OPcache keeps the compiled bytecode in shared memory, so compilation happens once. It's the single biggest free win in production.

```
opcache.enable=1
opcache.memory_consumption=256      ; MB for cached bytecode
opcache.max_accelerated_files=20000 ; raise for large codebases
opcache.validate_timestamps=0       ; prod: don't stat files — clear
on deploy
```

Gotcha: with `validate_timestamps=0`, PHP won't notice changed files — you must clear OPcache (or restart FPM) on every deploy, or you serve stale code.

FPM pool sizing

THE KEY DIALS

```
pm = dynamic
pm.max_children = 20
pm.start_servers = 4
pm.min_spare_servers = 2
pm.max_spare_servers = 6
pm.max_requests = 500
```

HOW TO SIZE IT

`max_children` $\approx$ available RAM $\div$ avg process memory. Too high $\rightarrow$ swapping (worse than queueing).
`max_requests` recycles workers to curb memory leaks. More workers $\neq$ more speed if the DB is the bottleneck.

`memory_limit` is per request, not per server — one request trying to load a million rows hits it and dies (by design). Raising it hides the real problem; stream the data instead (see §29).

32 · Local vs Staging vs Production

WHY IT MATTERS

“It works on my machine” is the oldest bug in the world. The senior answer is environment parity — making local, staging and prod as identical as possible — and keeping the differences in config, never in code.

WHY THEY DIFFER

- Different PHP version or extensions
- Different DB version / collation
- Far more data, real concurrency in prod
- Debug on locally, off in prod
- File paths, timezones, locale

HOW TO KEEP PARITY

- Docker — same image everywhere (\$6)
- ``.env`` per environment — config, not code
- Same migrations run in every env
- Staging mirrors prod (anonymised data)
- Pin versions in `composer.json` + lock file

Config belongs in the environment

The Twelve-Factor rule: anything that differs between environments — credentials, hostnames, feature flags, debug level — lives in environment variables, never committed. The same build artifact ships to every environment; only the config changes.

```
// Read from the environment, with a safe default
$debug = (bool) ($_ENV['APP_DEBUG'] ?? false);
$dbHost = $_ENV['DB_HOST'] ?? '127.0.0.1';

// In Laravel/Symfony this is wrapped:
$debug = config('app.debug'); // env('APP_DEBUG') under the hood
```

The prod-only settings that matter: `display_errors=Off` (log, don't show), `APP_DEBUG=false`, OPcache timestamps off, real mail driver, HTTPS-only cookies. A debug page leaking a stack trace is a security bug.

33 · Observability & Debugging Prod

WHY IT MATTERS

You can't attach a step-debugger to production. The senior skill is making the system tell you what's wrong — through logs, metrics and traces — and having a calm method when it does break.

THE THREE PILLARS

- Logs — discrete events (structured JSON, with context)
- Metrics — numbers over time (req/s, error rate, p95 latency)
- Traces — one request's path across services (APM)

LOG WELL

- Use PSR-3 levels (debug → emergency) meaningfully
- Structured logs (JSON) you can actually query
- Add a correlation / request ID to follow one request
- Never log secrets, tokens or full card numbers

```
// Structured, contextual logging (Monolog / PSR-3)
$logger->error('Payment failed', [
    'order_id' => $order->id,
    'gateway'   => 'stripe',
    'request_id' => $requestId,
    // no card data, no secrets
]);
```

A calm method when prod breaks

1. Stop the bleeding — roll back or feature-flag off first; diagnose after.
2. Check what changed — recent deploy? traffic spike? dependency down?
3. Read the signals — error rate, latency graph, logs around the spike.
4. Reproduce in staging, fix, add a test so it can't recur.
5. Blameless postmortem — fix the system, not the person.

Say this: “In production I lean on structured logs, metrics and tracing rather than guesswork. My first instinct in an incident is to roll back and stop the bleeding, then diagnose calmly from the signals.”

Quick Reference · Phrases to Use

Drop these into your answers — they signal you think about why, not just how.

Topic	Key phrase
DI	Loose coupling and testability — program to interfaces, not implementations.
SOLID	Each principle keeps classes small, focused and easy to change.
N+1	Eager load relationships rather than lazy loading in a loop.
Slow queries	First thing I'd do is run EXPLAIN and check for a full table scan.
Composition	I prefer composing behaviour through injected dependencies over deep inheritance.
== vs ===	I default to === and strict_types — coercion should be deliberate.
Transactions	Wrap related writes so they all commit or all roll back — ACID.
Caching	Cache-aside with a TTL; the hard part is invalidation on write.
Shared-nothing	Each request starts fresh, so state lives in Redis or the DB — and we scale out.
Queues	Push slow work to a background worker; keep jobs idempotent.
Multi-tenancy	Scope every query by tenant by default, not by memory.
Testing	If something is hard to test, the design usually needs revisiting.
Indexes	Covering and composite, equality-first — verify with EXPLAIN before adding.
Slow page	Measure and profile first; usually it's query count, not raw CPU.
Connections	Connections scale with FPM workers — pool with ProxySQL / PgBouncer.
Big result sets	Chunk or stream with a cursor; never load a million rows into memory.
PHP upgrades	Free performance — stay on a supported version, automate with Rector.
OPcache	On in prod, timestamps off, clear it on every deploy.
Env parity	Differences live in config, never code — same artifact, different .env.
Observability	Logs, metrics and traces — make production tell you what's wrong.

You've got this, Chris

The fact you can build production systems, ship real products, and explain your thinking clearly puts you ahead of most candidates. Read the Why it matters boxes once more, then go in and have a conversation — not an exam.